Práctica N°2: Medicion de Corriente con ajuste de PWM visualizado en OLED mediante comunicación Maestro-Esclavo con protocolo CAN

Índice

1. Objetivos **3**
- 1.1. Objetivo general . . . 3
- 1.2. Objetivos específicos . . . 3

2. Guía de configuración del Maestro en STM32CubeMX (`.ioc`) **4**
- 2.1. Pines en el ioc . . . 4
- 2.2. USART2 — Telemetría y comandos desde PC . . . 8
- 2.3. SPI2 + GPIO — Sensor de corriente Hall (TLI4970) . . . 8
- 2.4. TIM6 — Base de tiempos estable . . . 9

3. Explicación del código del Maestro **10**
- 3.1. Includes . . . 10
- 3.2. Variables globales y *handles* de periféricos . . . 10
- 3.3. Prototipos de funciones de inicialización . . . 11
- 3.4. Callback por interrupción de UART (recepción de comandos) . 12
- 3.5. `main()` — Secuencia de arranque y lazo principal . . . 13

4. Configuración del Esclavo en STM32CubeMX (`.ioc`) **16**

5. Explicación del código del Esclavo **19**
- 5.1. Macros y constantes de aplicación . . . 19
- 5.2. Variables globales y *handles* . . . 19
- 5.3. Prototipos relevantes . . . 20
- 5.4. Configuración de filtros CAN y callback de recepción . . . 20
- 5.5. Rutina de actualización de la OLED . . . 22

5.6. `main()` — Arranque, inicialización y bucle principal 23

6. Resultados **25**

6.1. Conexión física del sistema 25

1. Objetivos

1.1. Objetivo general

Diseñar, implementar y validar una arquitectura maestro–esclavo basada en comunicación **CAN** sobre la plataforma *STM32F407G-DISC1*, en la que el **maestro** mida en tiempo real la corriente del motor mediante un sensor Hall y la publique por CAN, mientras que el **esclavo** reciba dichos datos para visualizarlos en una pantalla OLED y actúe sobre el driver de motor mediante control PWM; adicionalmente, habilitar en el maestro una interfaz por **UART2** para telemetría y para el envío de comandos de control (`A/a` para incrementar y `d/D` para decrementar el PWM del motor del esclavo).

1.2. Objetivos específicos

- **Configurar el bus CAN** del STM32 (bit timing, modo normal/loopback para pruebas, filtros de aceptación e IDs) y definir un *frame set* mínimo (IDs, frecuencia de publicación y formato de payload) para corriente, estado y comandos.
- **Adquirir y escalar la corriente** en el maestro desde el sensor Hall (vía SPI u otra interfaz según el módulo empleado), implementando linealización, conversión a unidades físicas (A) y verificación básica de rango.
- **Implementar telemetría por UART2** en el maestro para imprimir corriente, estado del sistema y confirmaciones de comandos; mapear los caracteres `A/a` y `d/D` a mensajes CAN de "*incremento/decremento de PWM*".
- **Recibir y procesar tramas CAN** en el esclavo para actualizar el *set-point* de PWM del motor y mantener sincronizado el estado mostrado en la pantalla.
- **Controlar el motor por PWM** en el esclavo (TIM adecuado) mediante el driver *TB6549FG* u otro equivalente, garantizando una resolución y frecuencia de conmutación apropiadas para la aplicación.
- **Visualizar en OLED** (p. ej., SSD1351) en el esclavo las magnitudes relevantes: corriente instantánea, porcentaje de PWM, ID/estado del bus, y mensajes de alerta.

- **Robustecer la comunicación**: manejo de errores CAN (TEC/REC, *error frames*), reintentos y estados *bus-off*; validaciones de integridad de payload y *timeouts* en el esclavo ante pérdida de telegramas.
- **Seguridad funcional básica**: límites de PWM y corriente (clamp y/o apagado) para proteger el motor y la electrónica ante sobrecorriente o comandos inválidos.
- **Plan de pruebas y métricas**: verificar latencia extremo a extremo (medición UART→CAN→PWM), tasa de actualización, ocupación del bus y comportamiento ante fallos (desconexión del maestro, ruido, colisiones).
- **Documentación reproducible**: registrar mapeos de pines, parámetros de CAN, esquema de IDs, formato de tramas, versiones de firmware y pasos de compilación/descarga para facilitar replicación en laboratorio.

2. Guía de configuración del Maestro en STM32CubeMX (`.ioc`)

2.1. Pines en el ioc

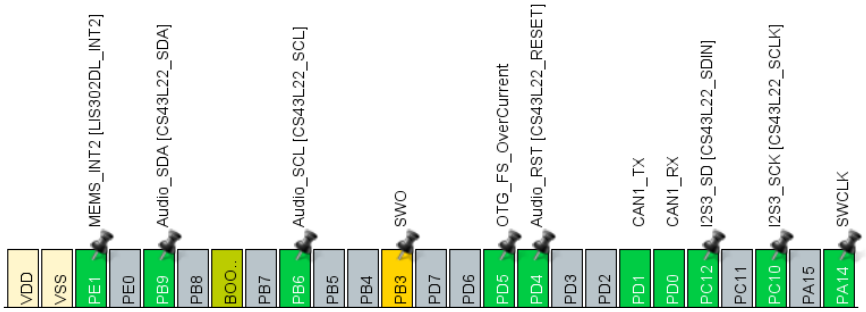


Figura 1: Vista superior (lado norte) del STM32F407G-DISC1 del maestro.

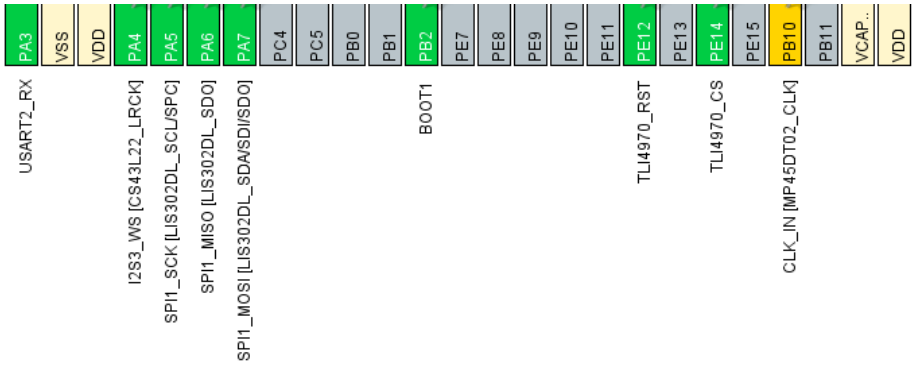


Figura 2: Vista inferior (lado sur).

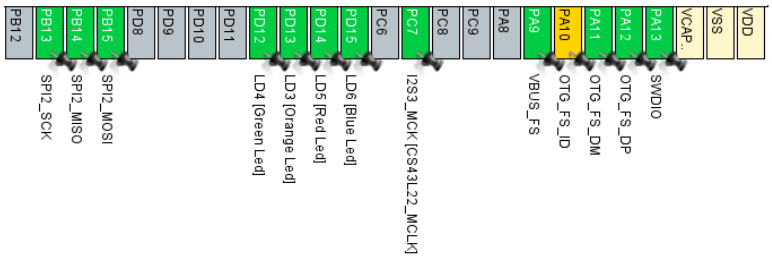


Figura 3: Vista lateral (este).

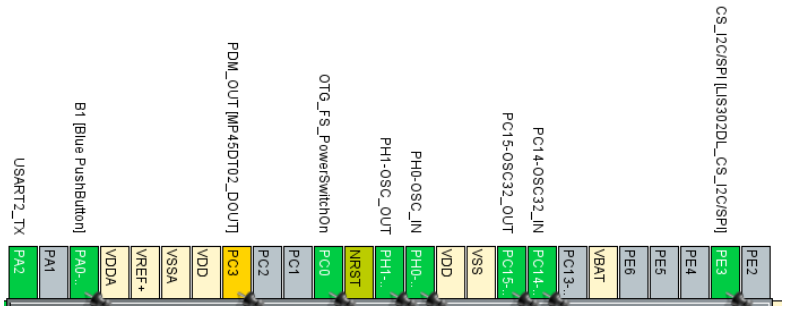


Figura 4: Vista lateral (oeste).

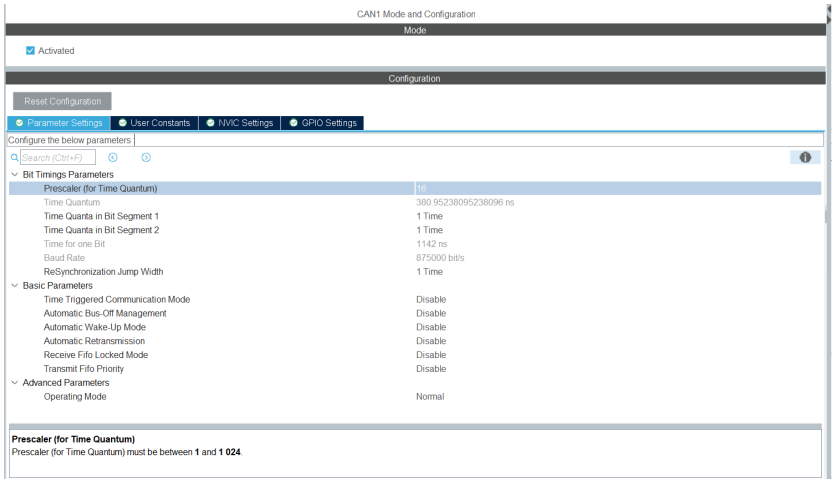


Figura 5: STM32CubeMX: configuración de **CAN1** (modo Normal, temporización nominal)

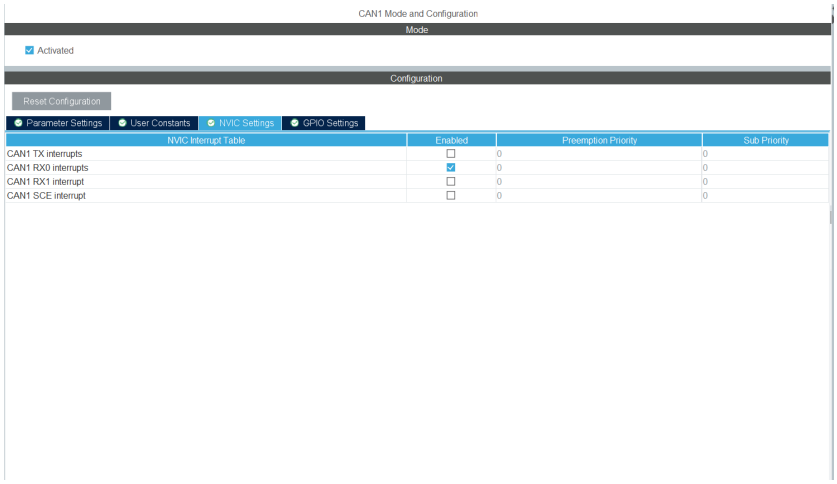


Figura 6: STM32CubeMX: configuración de **CAN1** (modo Normal, temporización nominal)

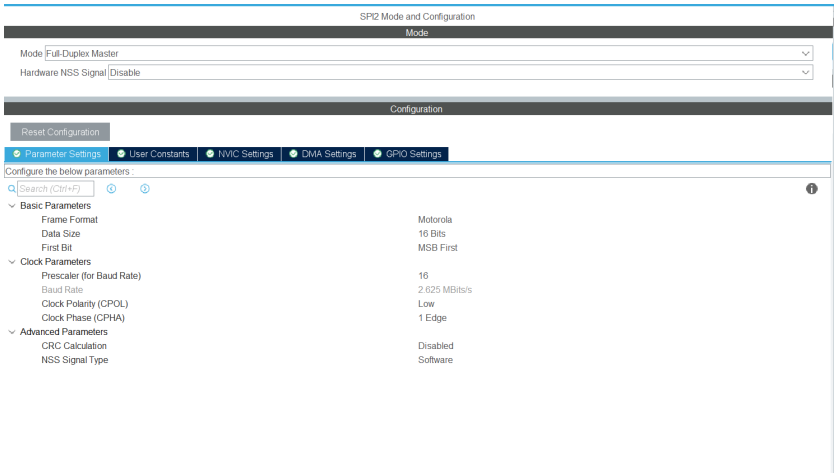


Figura 7: STM32CubeMX: **SPI2** en modo maestro.

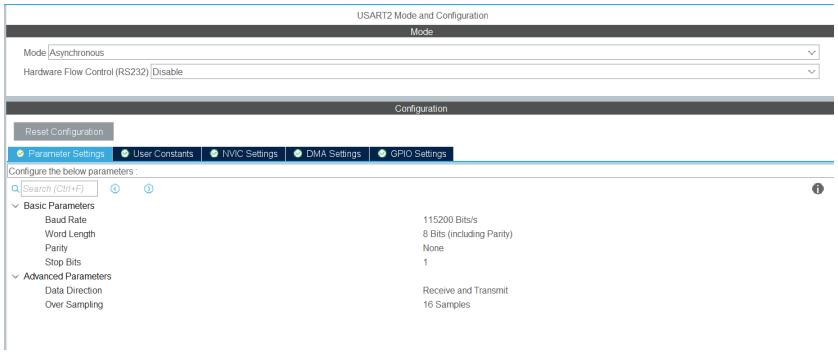


Figura 8: STM32CubeMX: **USART2**.

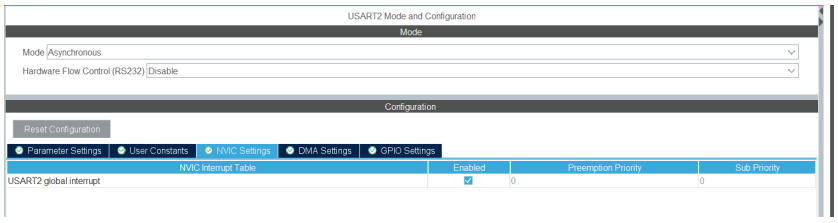


Figura 9: STM32CubeMX: **USART2**.

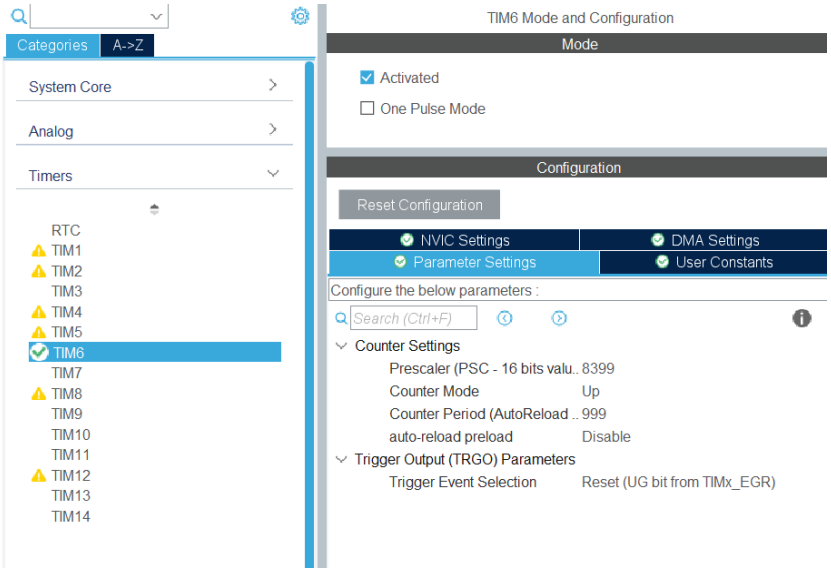


Figura 10: STM32CubeMX: **TIM6**.

2.2. USART2 — Telemetría y comandos desde PC

Propósito: imprimir corriente y leer las teclas `A/a` (subir PWM) y `D/d` (bajar PWM).

Configuración básica

1. Habilita **USART2**.
2. Parámetros: **115200 baud**, 8 N 1, sin control de flujo (`None`).
3. **Modo**: TX y RX.
4. En *NVIC*, habilita la **interrupción de USART2** (para recepción por IT).

Asignación de pines (AF7)

- **PA2** → USART2_TX (AF7)
- **PA3** → USART2_RX (AF7)

2.3. SPI2 + GPIO — Sensor de corriente Hall (TLI4970)

Propósito: adquirir la corriente del motor.

SPI2 (AF5)

1. Habilita **SPI2** como **Master**.
2. Parámetros:
   - `Data Size` = **16 bits**
   - `CPOL` = Low, `CPHA` = 1Edge
   - `NSS` = Software, `First Bit` = MSB
   - `BaudRate Prescaler` = **16**
3. Pines:
   - **PB13** = SPI2_SCK (AF5)
   - **PB14** = SPI2_MISO (AF5)
   - **PB15** = SPI2_MOSI (AF5)

GPIO de control del sensor

- **PE5** = `TLI4970_CS` → **GPIO Output** (Push–Pull, Low speed)
- **PE10** = `TLI4970_RST` → **GPIO Output** (Push–Pull, Low speed)

2.4. TIM6 — Base de tiempos estable

Propósito: temporizar tareas periódicas (telemetría, publicación CAN) con jitter bajo.

Configuración sugerida

- **TIM6** como *time base*. Prescaler y Period para **10 Hz** (100 ms por actualización).
- Habilita interrupción del TIM6 si quieres disparar tareas periódicas sin `HAL_Delay`.

3. Explicación del código del Maestro

3.1. Includes

```
// HAL y middlewares generados
#include "main.h"
#include "usb_host.h"

// Librer as est ndar C
#include <stdio.h>
#include <string.h>
```

¿Para qué sirven?

- `main.h` / `usb_host.h`: definiciones del hardware y middlewares generados por CubeMX.
- `stdio.h` / `string.h`: formateo de textos (p.ej., `sprintf`) y manejo de buffers para UART.

3.2. Variables globales y *handles* de periféricos

```
// Handles de perif ricos (HAL)
CAN_HandleTypeDef hcan1;

I2C_HandleTypeDef hi2c1;
I2S_HandleTypeDef hi2s3;

SPI_HandleTypeDef hspi1;
SPI_HandleTypeDef hspi2;

TIM_HandleTypeDef htim6;

UART_HandleTypeDef huart2;

// Variables de aplicaci n
uint8_t uart_rx_data;                    // Byte recibido
   por UART2
volatile uint8_t motor_duty_cycle = 0; // Consigna de
   PWM (0..100%)
```

Idea clave.

- Los *handles* encapsulan el estado de cada periférico; se pasan a las funciones HAL.
- `uart_rx_data` es un buffer de 1 byte para comandos.
- `motor_duty_cycle` es `volatile` porque puede cambiar dentro de una interrupción.

3.3. Prototipos de funciones de inicialización

```
1  void SystemClock_Config(void);
2  static void MX_GPIO_Init(void);
3  static void MX_I2C1_Init(void);
4  static void MX_I2S3_Init(void);
5  static void MX_SPI1_Init(void);
6  static void MX_CAN1_Init(void);
7  static void MX_SPI2_Init(void);
8  static void MX_TIM6_Init(void);
9  static void MX_USART2_UART_Init(void);
10 void MX_USB_HOST_Process(void);
```

Lectura guiada. Cada `MX_..._Init` prepara un periférico (modo, pines, velocidades, IRQ). En esta práctica nos interesan especialmente **CAN1**, **SPI2** (sensor Hall) y **USART2** (telemetría/comandos). **TIM6** es útil si luego quieres periodicidad precisa sin `HAL_Delay`.

3.4. Callback por interrupción de UART (recepción de comandos)

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    if (uart_rx_data == 'A' || uart_rx_data == 'a') {
      if (motor_duty_cycle <= 95) motor_duty_cycle += 5;
      else motor_duty_cycle = 100;
    }
    else if (uart_rx_data == 'D' || uart_rx_data == 'd')
        {
      if (motor_duty_cycle >= 5) motor_duty_cycle -= 5;
      else motor_duty_cycle = 0;
    }

    // Feedback por UART
    char command_buf[60];
    sprintf(command_buf, "Comando: %s. Nueva velocidad
       objetivo: %d%%\r\n",
            (uart_rx_data == 'A' || uart_rx_data == 'a')
                ? "Aumentar" : "Disminuir",
            motor_duty_cycle);
    HAL_UART_Transmit(&huart2, (uint8_t*)command_buf,
       strlen(command_buf), 100);

    // Publicar consigna por CAN (ID 0x124, DLC=1)
    CAN_TxHeaderTypeDef TxHeader;
    uint32_t TxMailbox;
    uint8_t speed_data[1] = { motor_duty_cycle };
    TxHeader.StdId = 0x124;
    TxHeader.RTR   = CAN_RTR_DATA;
    TxHeader.IDE   = CAN_ID_STD;
    TxHeader.DLC   = 1;
    HAL_CAN_AddTxMessage(&hcan1, &TxHeader, speed_data,
       &TxMailbox);

    // Re-armar la recepci n por interrupci n (
       escuchar el siguiente byte)
    HAL_UART_Receive_IT(&huart2, &uart_rx_data, 1);
  }
}
```

Puntos importantes.

- El callback se ejecuta cuando llega **un byte** por **USART2**.
- Mapa de comandos: `A/a` $\rightarrow +5\,\%$, `D/d` $\rightarrow -5\,\%$ (saturado a [0,100]).
- Se informa por UART y se envía la nueva consigna por CAN (ID `0x124`).
- Crítico: volver a llamar a `HAL_UART_Receive_IT` para seguir recibiendo.

3.5. `main()` — Secuencia de arranque y lazo principal

```
int main(void)
{
  /* Reset de HAL y SysTick */
  HAL_Init();

  /*  rbol  de relojes del sistema */
  SystemClock_Config();

  /* Inicializaci n de perif ricos */
  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_I2S3_Init();
  MX_SPI1_Init();
  MX_USB_HOST_Init();
  MX_CAN1_Init();
  MX_SPI2_Init();
  MX_TIM6_Init();
  MX_USART2_UART_Init();

  /* UART2: recibir por interrupci n (1 byte) */
  HAL_UART_Receive_IT(&huart2, &uart_rx_data, 1);

  /* CAN1: arrancar el bus */
  if (HAL_CAN_Start(&hcan1) != HAL_OK) {
      Error_Handler();
  }

  /* Bucle principal */
  while (1)
  {
```

```
31      MX_USB_HOST_Process();  // No interfiere si no usas
           USB
32
33      /* --- Adquisici n y filtrado de corriente (SPI2)
           --- */
34      const int NUM_SAMPLES = 100;
35      int32_t sum_current_value = 0;
36
37      for (int i = 0; i < NUM_SAMPLES; i++) {
38          uint16_t raw_spi_frame = 0;
39
40          // Control manual de CS (GPIO)
41          HAL_GPIO_WritePin(TLI4970_CS_GPIO_Port,
               TLI4970_CS_Pin, GPIO_PIN_RESET);
42          HAL_Delay(1);
43          HAL_SPI_Receive(&hspi2, (uint8_t *)&
               raw_spi_frame, 1, 100);
44          HAL_GPIO_WritePin(TLI4970_CS_GPIO_Port,
               TLI4970_CS_Pin, GPIO_PIN_SET);
45
46          // Swap de bytes (endianness)
47          raw_spi_frame = (raw_spi_frame << 8) | (
               raw_spi_frame >> 8);
48
49          // Extraer 13 bits con signo + extensi n de
               signo
50          int16_t current_value_13bit = raw_spi_frame & 0
               x1FFF;
51          if (current_value_13bit & 0x1000) {
52              current_value_13bit |= 0xE000;
53          }
54
55          sum_current_value += current_value_13bit;
56      }
57
58      // Promedio y escala a miliamperios (12.5 mA/LSB)
59      float current_mA = (float)(sum_current_value /
           NUM_SAMPLES) * 12.5f;
60
61      /* --- Transmisi n por CAN: corriente (ID 0x123, 2
           bytes big-endian) --- */
62      CAN_TxHeaderTypeDef TxHeader;
63      uint32_t TxMailbox;
```

```
64      uint8_t can_data[2];
65      TxHeader.StdId = 0x123;
66      TxHeader.RTR   = CAN_RTR_DATA;
67      TxHeader.IDE   = CAN_ID_STD;
68      TxHeader.DLC   = 2;
69
70      int16_t current_to_send = (int16_t)current_mA;
71      can_data[0] = (uint8_t)(current_to_send >> 8);
72      can_data[1] = (uint8_t)(current_to_send);
73      HAL_CAN_AddTxMessage(&hcan1, &TxHeader, can_data, &
            TxMailbox);
74
75      /* --- Telemetr a UART --- */
76      char uart_buf[50];
77      sprintf(uart_buf, "Corriente: %.2f mA\r\n",
            current_mA);
78      HAL_UART_Transmit(&huart2, (uint8_t*)uart_buf,
            strlen(uart_buf), 100);
79
80      /* Periodicidad simple (0.5 s). Mejora futura: TIM6
            + IRQ */
81      HAL_Delay(500);
82    }
83  }
```

Cómo leer este flujo.

1. Arranque de HAL y relojes; luego inicialización de periféricos.
2. Se habilita la recepción por interrupción en UART2 y se pone CAN1 operativo.
3. En el `while(1)`:
   a) Se adquieren 100 muestras por SPI2 con control manual de CS y corrección de *endianness*.
   b) Se extraen 13 bits con signo, se promedian y se escalan a mA.
   c) Se envía la corriente por CAN (ID `0x123`) y se imprime por UART.
   d) Se espera 500 ms (versión simple y entendible para iniciar).

4. Configuración del Esclavo en STM32CubeMX (`.ioc`)

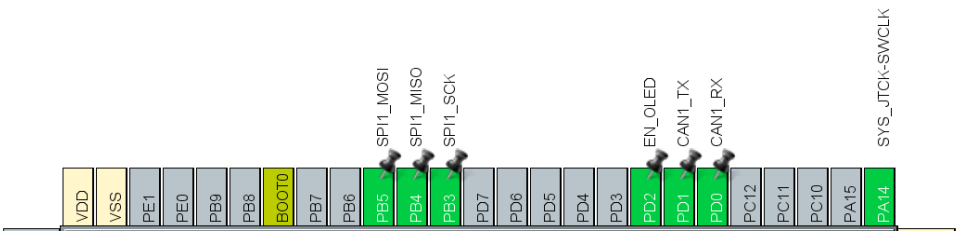


Figura 11: Esclavo — vista superior (lado norte).

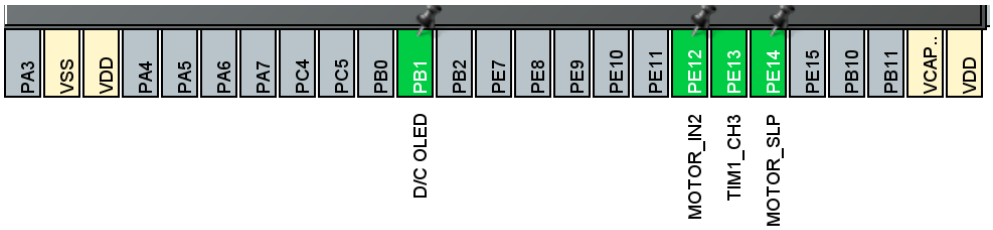


Figura 12: Esclavo — vista inferior (lado sur).

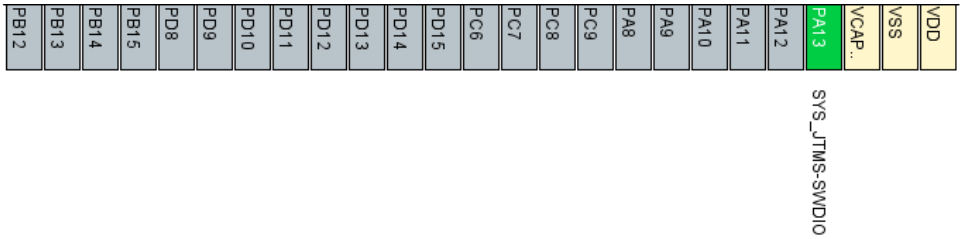


Figura 13: Esclavo — vista lateral (este).

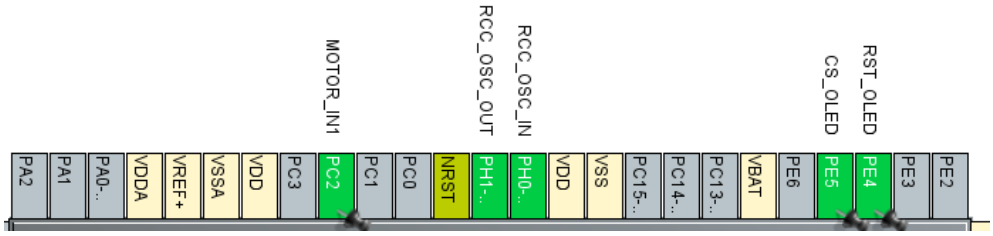


Figura 14: Esclavo — vista lateral (oeste)s.

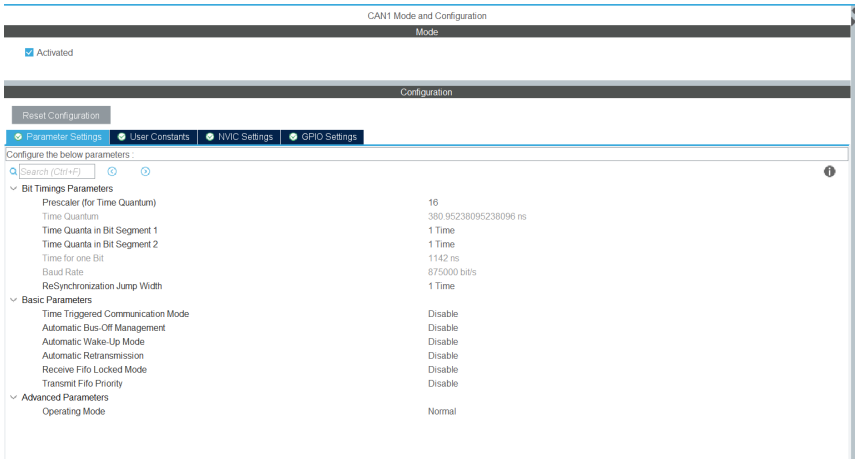


Figura 15: STM32CubeMX — Esclavo: configuración de **CAN1** en modo normal, filtros y notificaciones de recepción activas.

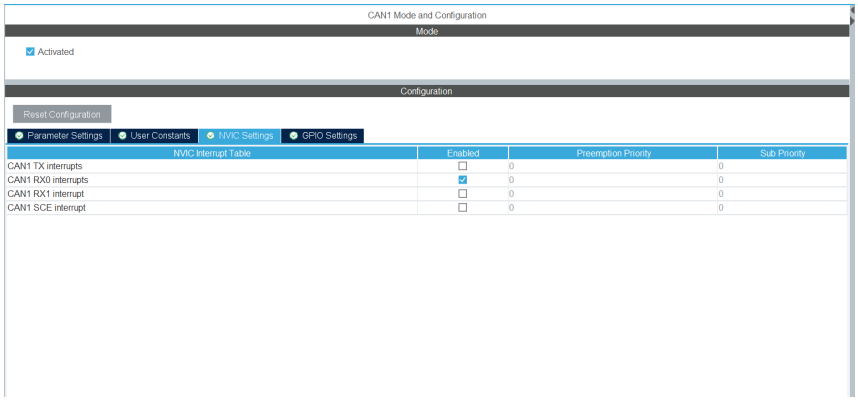


Figura 16: STM32CubeMX — Esclavo: configuración de **CAN1** en modo normal, filtros y notificaciones de recepción activas.

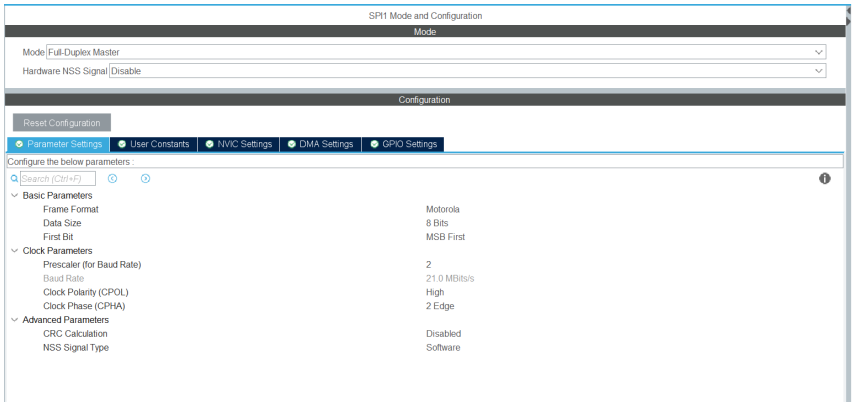


Figura 17: STM32CubeMX — Esclavo: **SPI1** configurado para la **OLED SSD1351**. Ajustes de modo maestro y parámetros de transferencia.

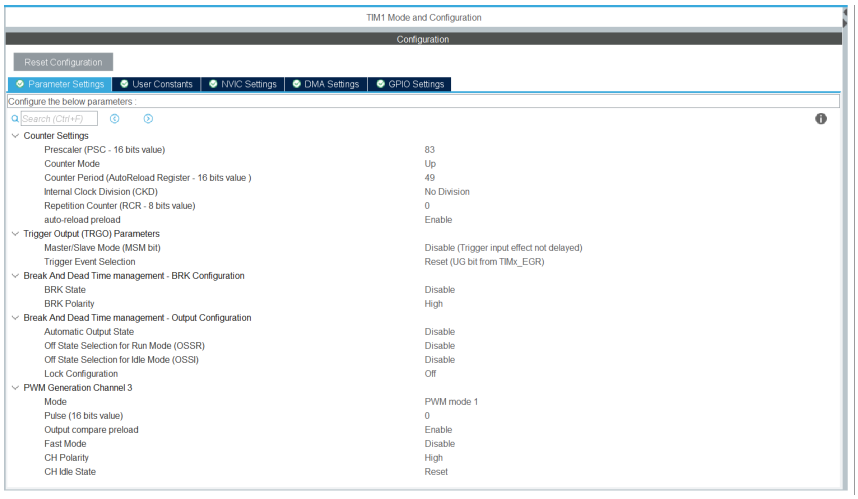


Figura 18: STM32CubeMX — Esclavo: **TIM1** en **PWM** para el control de velocidad del motor mediante el driver.

5. Explicación del código del Esclavo

5.1. Macros y constantes de aplicación

```
/* USER CODE BEGIN PD */
#define CAN_ID_CURRENT 0x123
#define CAN_ID_SPEED   0x124

#define BLACK   0x0000
#define WHITE   0xFFFF
#define RED     0xF800
#define GREEN   0x07E0
#define BLUE    0x001F
#define YELLOW  0xFFE0
/* USER CODE END PD */
```

Lectura guiada.

- `CAN_ID_CURRENT` y `CAN_ID_SPEED`: identificadores de las tramas que el nodo debe aceptar (corriente y consigna de velocidad).
- Constantes de color para la OLED (formato RGB565): facilitan textos legibles y consistentes.

5.2. Variables globales y *handles*

```
/* Private variables
   -----------------------------------*/
CAN_HandleTypeDef hcan1;

SPI_HandleTypeDef hspi1;

TIM_HandleTypeDef htim1;

/* USER CODE BEGIN PV */
volatile int16_t g_current_mA = 0;
volatile uint8_t g_motor_speed_percent = 0;
volatile _Bool g_update_display_flag = 0;
/* USER CODE END PV */
```

Puntos importantes.

- **Estado de periféricos**: `hcan1`, `hspi1` (OLED), `htim1` (PWM motor).

- **Volátiles**: las variables compartidas con interrupciones (`g_current_mA`, `g_motor_speed_percent`, `g_update_display_flag`) son `volatile` para evitar que el compilador "optimice" lecturas/escrituras asíncronas.

5.3. Prototipos relevantes

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_CAN1_Init(void);
static void MX_SPI1_Init(void);
static void MX_TIM1_Init(void);
/* USER CODE BEGIN PFP */
void Update_OLED_Display(void);
static void CAN_Filter_Config(void);
/* USER CODE END PFP */
```

Idea clave.

- Las funciones `MX_..._Init` inicializan periféricos.
- `CAN_Filter_Config`: configura filtros para recibir **solo** lo que interesa (IDs de corriente y velocidad).
- `Update_OLED_Display`: encapsula la lógica de refresco de pantalla para no mezclarla con las ISR.

5.4. Configuración de filtros CAN y callback de recepción

```
/* USER CODE BEGIN 4 */
static void CAN_Filter_Config(void)
{
  CAN_FilterTypeDef canfilterconfig;
  canfilterconfig.FilterActivation     =
     CAN_FILTER_ENABLE;
  canfilterconfig.FilterBank           = 0;
  canfilterconfig.FilterFIFOAssignment = CAN_RX_FIFO0;
  canfilterconfig.FilterMode           =
     CAN_FILTERMODE_IDLIST;
  canfilterconfig.FilterScale          =
     CAN_FILTERSCALE_32BIT;
  canfilterconfig.SlaveStartFilterBank = 14;
```

```
11    canfilterconfig.FilterIdHigh         = CAN_ID_CURRENT
         << 5;
12    canfilterconfig.FilterIdLow          = 0x0000;
13    canfilterconfig.FilterMaskIdHigh     = CAN_ID_SPEED <<
          5;
14    canfilterconfig.FilterMaskIdLow      = 0x0000;
15    if(HAL_CAN_ConfigFilter(&hcan1, &canfilterconfig) !=
         HAL_OK) { Error_Handler(); }
16  }
17
18  void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef
        *hcan)
19  {
20      CAN_RxHeaderTypeDef RxHeader;
21      uint8_t RxData[8];
22      if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &
           RxHeader, RxData) == HAL_OK)
23      {
24          if (RxHeader.StdId == CAN_ID_CURRENT && RxHeader
               .DLC == 2)
25          {
26              g_current_mA = (int16_t)((RxData[0] << 8) |
                   RxData[1]);
27          }
28          else if (RxHeader.StdId == CAN_ID_SPEED &&
               RxHeader.DLC == 1)
29          {
30              g_motor_speed_percent = RxData[0];
31              if (g_motor_speed_percent > 100)
                   g_motor_speed_percent = 100;
32              uint32_t pulse_value = (uint32_t)(
                   g_motor_speed_percent * 49) / 100;
33              __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3,
                    pulse_value);
34          }
35          g_update_display_flag = 1;
36      }
37  }
38  /* USER CODE END 4 */
```

Explicación.

- **Filtro por lista de IDs** (`IDLIST`): el nodo acepta dos identificadores concretos (corriente y consigna). Esto simplifica el procesamiento y

reduce tráfico indeseado.

- **Callback de recepción**:
  - Si llega `CAN_ID_CURRENT` con `DLC=2`, se reconstruye el entero con orden *big-endian* y se guarda en `g_current_mA`.
  - Si llega `CAN_ID_SPEED` con `DLC=1`, se actualiza el porcentaje de velocidad y se traduce a **ciclo útil** del PWM (`TIM1, CH3`) ajustando el `CCR`.
  - `g_update_display_flag`=1 notifica al *main loop* que la pantalla debe refrescarse.

5.5. Rutina de actualización de la OLED

```
void Update_OLED_Display(void)
{
    char buffer[24];
    float estimated_rpm = (g_motor_speed_percent / 100.0
       f) * 60.0f;

    SSD1351_Clear();

    snprintf(buffer, sizeof(buffer), "I: %.2f A",
       g_current_mA / 1000.0f);
    SSD1351_DrawText(10, 10, buffer, GREEN, BLACK, 2);

    snprintf(buffer, sizeof(buffer), "Vel: %d %%",
       g_motor_speed_percent);
    SSD1351_DrawText(10, 40, buffer, BLUE, BLACK, 2);

    snprintf(buffer, sizeof(buffer), "RPM: %.1f",
       estimated_rpm);
    SSD1351_DrawText(10, 70, buffer, WHITE, BLACK, 2);
}
```

Puntos Claves.

- La función reúne todas las llamadas a la librería de la OLED para mantener el `while(1)` limpio.
- Se convierten unidades: mA → A (para lectura clara) y se muestra una RPM estimada proporcional al % de PWM (modelo simple útil para prácticas).

5.6. main() — Arranque, inicialización y bucle principal

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_CAN1_Init();
  MX_SPI1_Init();
  MX_TIM1_Init();

  // --- Iniciar Perif ricos ---
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
  CAN_Filter_Config();
  HAL_CAN_Start(&hcan1);
  HAL_CAN_ActivateNotification(&hcan1,
     CAN_IT_RX_FIFO0_MSG_PENDING);

  // --- Activar Driver y Pantalla ---
  HAL_GPIO_WritePin(EN_OLED_GPIO_Port, EN_OLED_Pin,
     GPIO_PIN_SET);
  HAL_GPIO_WritePin(MOTOR_SLP_GPIO_Port, MOTOR_SLP_Pin,
     GPIO_PIN_SET);
  HAL_GPIO_WritePin(MOTOR_IN1_GPIO_Port, MOTOR_IN1_Pin,
     GPIO_PIN_SET);
  HAL_GPIO_WritePin(MOTOR_IN2_GPIO_Port, MOTOR_IN2_Pin,
     GPIO_PIN_RESET);

  // --- Inicializar Pantalla ---
  SSD1351_Init();
  SSD1351_Clear();
  SSD1351_DrawText(5, 50, "ESCLAVO␣LISTO", YELLOW, BLACK
     , 1);
  SSD1351_DrawText(15, 65, "Esperando␣CAN...", WHITE,
     BLACK, 1);
  HAL_Delay(1500);

  while (1)
  {
    if (g_update_display_flag)
    {
```

```
34            Update_OLED_Display();
35            g_update_display_flag = 0;
36        }
37    }
38 }
```

Lectura guiada.

1. **Inicialización**: HAL, relojes y periféricos (GPIO, CAN, SPI de la OLED, TIM para PWM).
2. **Puesta en marcha**:
   - PWM activo en el canal configurado.
   - Filtros CAN cargados, bus iniciado y notificaciones de RX habilitadas (FIFO0).
   - Señales de habilitación del driver y OLED activadas; pantalla inicializada y mensaje de estado.
3. **Bucle**:
   - El refresco de la OLED se hace sólo cuando hay nuevos datos (`g_update_display_flag`), evitando redibujar sin necesidad.
   - La actualización del PWM no está en el lazo: se realiza dentro del **callback CAN** para responder inmediatamente a las consignas.

6. Resultados

6.1. Conexión física del sistema

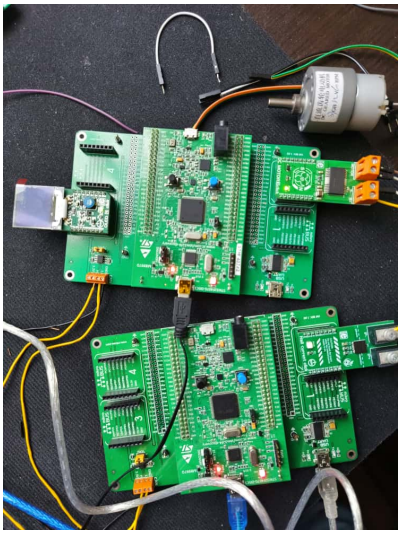


Figura 19: Maestro y Esclavo: cableado final. Se aprecia la interfaz hacia el transceptor CAN, conexión del sensor de corriente (SPI) y UART hacia el PC.

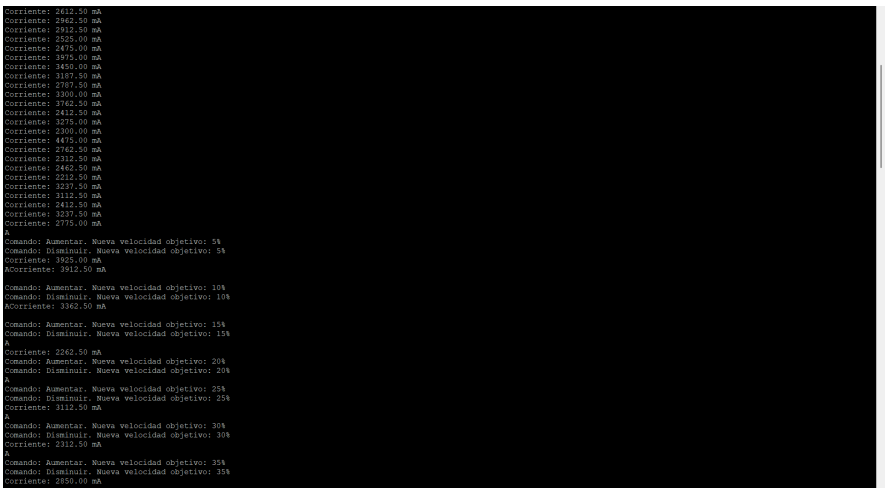


Figura 20: Sesión PuTTY: trazas periódicas de corriente y acuses al enviar `A/a` (sube PWM) y `D/d` (baja PWM).

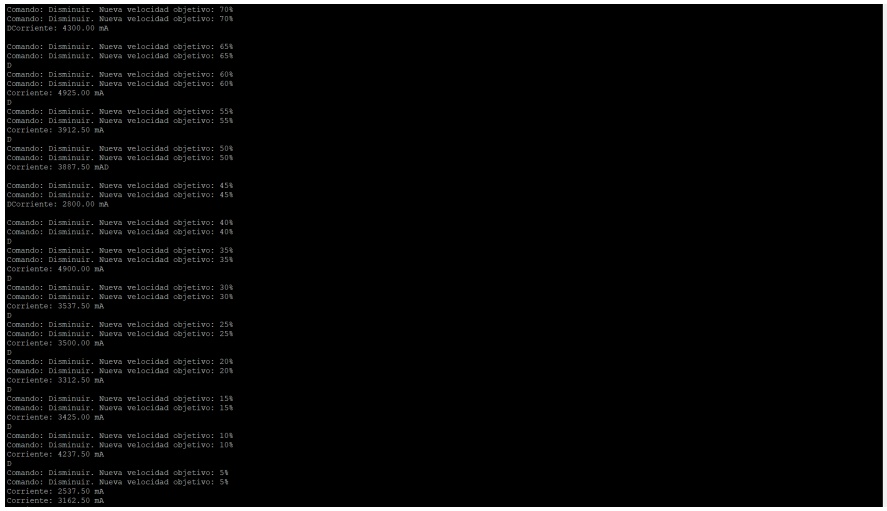


Figura 21: Sesión PuTTY: trazas periódicas de corriente y acuses al enviar `A/a` (sube PWM) y `D/d` (baja PWM).